94f176b0-7689-4f23-b5fa-ea161dba9be7

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'intel/torch-xpu-ops' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/torch-xpu-ops/torch-xpu-ops (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-18T19:49:57.959+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Fix arange error type for fractional step with integer output on XPU
</pr_title>

<pr_description>
Summary
Fix `arange_out_xpu` to raise `ValueError` (instead of `RuntimeError`) when a fractional step truncates to zero for an integer output tensor, matching the CUDA implementation.

Changes
- Include `<ATen/native/RangeUtils.h>` for `arange_check_bounds`.
- Replace manual `TORCH_CHECK` calls with

`arange_check_bounds(start, end, step)` for consistent double-precision validation.
- Add `TORCH_CHECK_VALUE(xstep != 0, "step must be nonzero")` in the `int64_t` code path (mirrors `RangeFactories.cu`).

Test
`python -m pytest -sxv test_ops_xpu.py -k test_errors_arange_xpu` now passes.

Fixes #3699
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
Pull request overview

Skill files read: `.github/skills/xpu-ops-pr-review/SKILL.md`.

This PR updates the XPU `arange_out_xpu` argument validation to better match CUDA semantics by centralizing bound checks in `arange_check_bounds(...)` and ensuring the integer-output fractional-step-to-zero case raises `ValueError`.

Changes:
- Added `#include <ATen/native/RangeUtils.h>` and switched to `arange_check_bounds(start, end, step)` for bounds/sign/finite validation.
- Added an `int64_t`-specific `TORCH_CHECK_VALUE(xstep != 0, "step must be nonzero")` to match CUDA’s error type for fractional steps truncating to zero.

</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>src/ATen/native/xpu/RangeFactories.cpp:47-52</file>
<original_commit>8584d5b</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
After replacing the pre-check TORCH_CHECKs with arange_check_bounds, the local variables xstart/xend/xstep are no longer used for most scalar_t instantiations (only the int64_t constexpr branch uses them). CI enables -Werror=unused-variable (see repo CMakeLists.txt:35), so this will fail compilation. Move these conversions into the branch where they’re needed (or otherwise mark them unused) so non-int64 instantiations don’t trigger unused-variable errors.
</comment_old>

</comment_thread>

<comment_thread>
<file>src/ATen/native/xpu/RangeFactories.cpp:50</file>
<original_commit>8584d5b</original_commit>

<comment_new>
<comment_id>3259555779</comment_id>
<author>@pbielak</author>
As `xstart`, `xend` and `xstep` are only used in the

```
if constexpr (std::is_same_v<scalar_t, int64_t>) {
```
branch, maybe they should be moved there?
</comment_new>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_new>
<comment_id>4311230059</comment_id>
<author>@BBBela</author>
I have the same concern as @pbielak regarding putting the `xstart`, `xend`, and `xstep` into the `constexpr` body where they are used. I would personally do a step further and move also the `using accscalar_t` alias into the `if constexpr` as it is also used only for those three variables - but it is not blocking in any case.

```
Beside that everything looks good!
</comment_new>
```

```
<comment_new>
<comment_id>4481440032</comment_id>
<author>@chuanqi129</author>
@copilot New UT failures detected in **op_ut** tests (1
total). Please check these new failures and analyze whether
they are caused by the PR changes.

**Important: Do NOT update the plan in the PR description
directly. Use reply comments to update the status.**

Job log: https://github.com/intel/torch-xpu-ops/actions/
runs/26033231815
Full new failure report: https://github.com/intel/torch-xpu-
ops/actions/runs/26033231815/artifacts/7067652345

**New failures:**

| Category | Class name | Test name | Status | Message |
| --- | --- | --- | --- | --- |
| op_ut | third_party.torch-xpu-
ops.test.xpu.functorch.test_vmap_xpu.TestVmapOperatorsOpInfo
XPU | test_op_has_batch_rule_count_nonzero_xpu_float32 |
failed | Unexpected success

Instructions:
1. Download the full new failure report artifact for
complete error messages
2. Analyze whether the failures are caused by PR changes
3. If caused by PR changes, fix those failures on this PR
branch directly
4. Add failing test cases with pytest commands in the PR
description for the reproducer job to use
5. Use reply comments to report analysis status - do NOT
modify the PR description plan directly
</comment_new>
```

</pr_comments>

</comments>

The last **1** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<system_reminder>
You have the capability to call multiple tools in a single response. For maximum efficiency, whenever you need to perform multiple independent operations, ALWAYS call tools simultaneously whenever the actions can be done in parallel rather than sequentially (e.g. multiple reads/edits to different files). Especially when exploring repository, searching, reading files, viewing directories, validating changes. For example, you can read 3 different files in parallel, or edit different files in parallel. However, if some tool calls depend on previous calls to inform dependent values like the parameters, do NOT call these tools in parallel and instead call them sequentially (e.g. reading shell output from a previous command should be sequential as it requires the sessionID).
</system_reminder>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

Implemented and pushed a surgical fix in `3e68faf`:

- Moved `using accscalar_t`, `xstart`, `xend`, and `xstep` into the `if constexpr (std::is_same_v<scalar_t, int64_t>)` branch in `/home/runner/work/torch-xpu-ops/torch-xpu-ops/src/ATen/native/xpu/RangeFactories.cpp`.
- This resolves the unused-variable risk for non-`int64_t` instantiations (`-Werror=unused-variable` concern).

CI failure investigation (run `26033231815`):
- I used GitHub Actions MCP (runs/jobs/artifacts/logs) and downloaded artifact `7067652345`.
- The reported test `test_op_has_batch_rule_count_nonzero_xpu_float32` is an **Unexpected success** from a pre-existing `xfail("count_nonzero")` in `test/xpu/functorch/test_vmap_xpu.py`.
- This PR only touches `RangeFactories.cpp` (`8584d5b`, `3e68faf`), so the op_ut failure is **not caused by this PR change**.

I replied to all actionable new comments:
- `3259555779`
- `4311230059`
- `4481440032`

Validation:
- `parallel_validation` passed (Code Review clean, CodeQL skipped as trivial refactor).
- Local pytest command could not run in this runner because `pytest` is not installed.

Skill files read:
- `.github/skills/xpu-issues-triaging/SKILL.md`
- `.github/skills/xpu-ops-pr-creation/SKILL.md`
- `.github/skills/xpu-ops-pr-review/references/torch-xpu-ops-review-notes.md`
- `.github/skills/xpu-ops-pr-review/references/review-checklist.md`
- `.github/skills/xpu-ops-pr-review/references/bc-guidelines.md`

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'intel/torch-xpu-ops' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/torch-xpu-ops/torch-xpu-ops (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-19T10:25:43.876+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Fix arange error type for fractional step with integer output on XPU
</pr_title>

<pr_description>
Summary
Fix `arange_out_xpu` to raise `ValueError` (instead of `RuntimeError`) when a fractional step truncates to zero for an integer output tensor, matching the CUDA implementation.

Changes
- Include `<ATen/native/RangeUtils.h>` for `arange_check_bounds`.
- Replace manual `TORCH_CHECK` calls with `arange_check_bounds(start, end, step)` for consistent double-precision validation.
- Add `TORCH_CHECK_VALUE(xstep != 0, "step must be nonzero")` in the `int64_t` code path (mirrors `RangeFactories.cu`).

Test
`python -m pytest -sxv test_ops_xpu.py -k test_errors_arange_xpu` now passes.

Fixes #3699
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
Pull request overview

Skill files read: `.github/skills/xpu-ops-pr-review/SKILL.md`.

This PR updates the XPU `arange_out_xpu` argument validation to better match CUDA semantics by centralizing bound checks in `arange_check_bounds(...)` and ensuring the integer-output fractional-step-to-zero case raises `ValueError`.

Changes:
- Added `#include <ATen/native/RangeUtils.h>` and switched to `arange_check_bounds(start, end, step)` for bounds/sign/finite validation.
- Added an `int64_t`-specific `TORCH_CHECK_VALUE(xstep != 0, "step must be nonzero")` to match CUDA’s error type for fractional steps truncating to zero.

</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread_resolved>
<file>src/ATen/native/xpu/RangeFactories.cpp:47-52</file>
<original_commit>8584d5b</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
After replacing the pre-check TORCH_CHECKs with arange_check_bounds, the local variables xstart/xend/xstep are no longer used for most scalar_t instantiations (only the int64_t constexpr branch uses them). CI enables -Werror=unused-variable (see repo CMakeLists.txt:35), so this will fail compilation. Move these conversions into the branch where they’re needed (or otherwise mark them unused) so non-int64 instantiations don’t trigger unused-variable errors.
</comment_old>

</comment_thread_resolved>

<comment_thread_resolved>
<file>src/ATen/native/xpu/RangeFactories.cpp:50</file>
<original_commit>8584d5b</original_commit>

<comment_old>
<author>@pbielak</author>
As `xstart`, `xend` and `xstep` are only used in the

```
if constexpr (std::is_same_v<scalar_t, int64_t>) {
```
branch, maybe they should be moved there?
</comment_old>

<comment_old>
<author>@copilot</author>
Done in 3e68faf: `xstart`, `xend`, and `xstep` are now declared only inside the `if constexpr (std::is_same_v<scalar_t, int64_t>)` block where they are used, so non-`int64_t` instantiations no longer see unused variables.
</comment_old>

</comment_thread_resolved>

</pr_reviews>

<pr_comments>

<comment_old>
<author>@BBBela</author>
I have the same concern as @pbielak regarding putting the `xstart`, `xend`, and `xstep` into the `constexpr` body where they are used. I would personally do a step further and move also the `using accscalar_t` alias into the `if constexpr` as it is also used only for those three variables - but it is not blocking in any case.
Beside that everything looks good!
</comment_old>

<comment_old>
<author>@chuanqi129</author>
@copilot New UT failures detected in **op_ut** tests (1 total). Please check these new failures and analyze whether they are caused by the PR changes.

Important: Do NOT update the plan in the PR description directly. Use reply comments to update the status.

Job log: https://github.com/intel/torch-xpu-ops/actions/runs/26033231815
Full new failure report: https://github.com/intel/torch-xpu-ops/actions/runs/26033231815/artifacts/7067652345

New failures:

Category	Class name	Test name	Status	Message
op_ut	third_party.torch-xpu-ops.test.xpu.functorch.test_vmap_xpu.TestVmapOperatorsOpInfoXPU	test_op_has_batch_rule_count_nonzero_xpu_float32	failed	Unexpected success

Instructions:
1. Download the full new failure report artifact for complete error messages
2. Analyze whether the failures are caused by PR changes
3. If caused by PR changes, fix those failures on this PR branch directly
4. Add failing test cases with pytest commands in the PR description for the reproducer job to use
5. Use reply comments to report analysis status - do NOT modify the PR description plan directly
</comment_old>

<comment_old>
<author>@copilot</author>

> I have the same concern as @pbielak regarding putting the `xstart`, `xend`, and `xstep` into the `constexpr` body where they are used. I would personally do a step further and move also the `using acc...

Addressed in 3e68faf: I moved `xstart`/`xend`/`xstep` into the `int64_t` `if constexpr` branch, and also moved the `using accscalar_t` alias there since it is only needed for those values.

> @copilot New UT failures detected in **op_ut** tests (1 total). Please check these new failures and analyze whether they are caused by the PR changes.

>
> **Important: Do NOT update the plan in the PR de...

I checked run `26033231815` and downloaded artifact `7067652345` (`new_ut_failure_list.csv`). The reported failure is `test_op_has_batch_rule_count_nonzero_xpu_float32` with `Unexpected success`, which comes from a pre-existing `xfail("count_nonzero")` in `test/xpu/functorch/test_vmap_xpu.py`. This PR only changes `src/ATen/native/xpu/RangeFactories.cpp` (`8584d5b`, follow-up `3e68faf`) and does not touch `count_nonzero`/functorch paths, so this failure is not caused by these PR changes. Reproducer command: `python -m pytest -x -vv test/xpu/functorch/test_vmap_xpu.py -k test_op_has_batch_rule_count_nonzero_xpu_float32`.
</comment_old>

<comment_new>
<comment_id>4316076984</comment_id>
<author>@BBBela</author>
Looks good to me in current shape!
Thanks!
</comment_new>

<comment_new>
<comment_id>4486804565</comment_id>
<author>@chuanqi129</author>
@copilot New UT failures detected in **op_ut** tests (1 total). Please check these new failures and analyze whether they are caused by the PR changes.

Important: Do NOT update the plan in the PR description directly. Use reply comments to update the status.

Job log: https://github.com/intel/torch-xpu-ops/actions/runs/26056824947
Full new failure report: https://github.com/intel/torch-xpu-ops/actions/runs/26056824947/artifacts/7080904551

New failures:

Category	Class name	Test name	Status	Message
op_ut	third_party.torch-xpu-ops.test.xpu.functorch.test_vmap_xpu.TestVmapOperatorsOpInfoXPU	test_op_has_batch_rule_count_nonzero_xpu_float32	failed	Unexpected success

Instructions:

1. Download the full new failure report artifact for complete error messages
2. Analyze whether the failures are caused by PR changes
3. If caused by PR changes, fix those failures on this PR branch directly
4. Add failing test cases with pytest commands in the PR description for the reproducer job to use
5. Use reply comments to report analysis status - do NOT modify the PR description plan directly
</comment_new>

</pr_comments>

</comments>

The last **2** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<system_reminder>
You have the capability to call multiple tools in a single response. For maximum efficiency, whenever you need to perform multiple independent operations, ALWAYS call tools simultaneously whenever

the actions can be done in parallel rather than sequentially (e.g. multiple reads/edits to different files). Especially when exploring repository, searching, reading files, viewing directories, validating changes. For example, you can read 3 different files in parallel, or edit different files in parallel. However, if some tool calls depend on previous calls to inform dependent values like the parameters, do NOT call these tools in parallel and instead call them sequentially (e.g. reading shell output from a previous command should be sequential as it requires the sessionID).
</system_reminder>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.